

Rapport Projet **GoatUdder**
Pipeline DevOps & Architecture Logicielle

GoatSoftware

21 juin 2026

Table des matières

Chapitre 1

Introduction

1.1 Contexte

Le projet **GoatUdder** est une plateforme web minimaliste de location de pis de chèvres. L'utilisateur peut louer un ou plusieurs pis pour une durée limitée et obtenir du lait de chèvre selon un modèle de location simple.

1.2 Objectifs DevOps

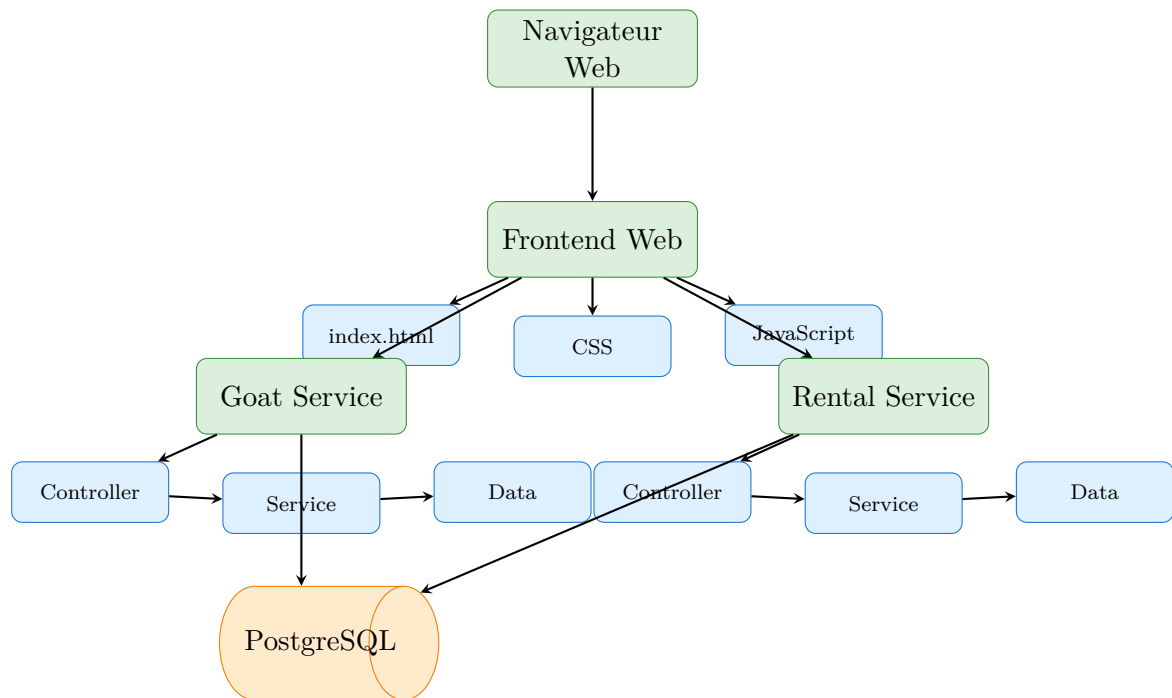
Développer une application en respectant les règles du DevOps :

- Dépôt Git structuré
- Pipeline CI (Continuous Integration) avec GitHub Actions
- Architecture logicielle en couches (Data, Services, Controller)
- Conteneurisation Docker de tous les services
- Tests de toutes les couches (unitaires, mocks web)
- Bonne couverture de code ($\geq 80\%$)
- Qualité logicielle élevée (SonarQube, ESLint)

Chapitre 2

Architecture Logicielle

2.1 Architecture globale



2.2 Architecture en couches (par service)



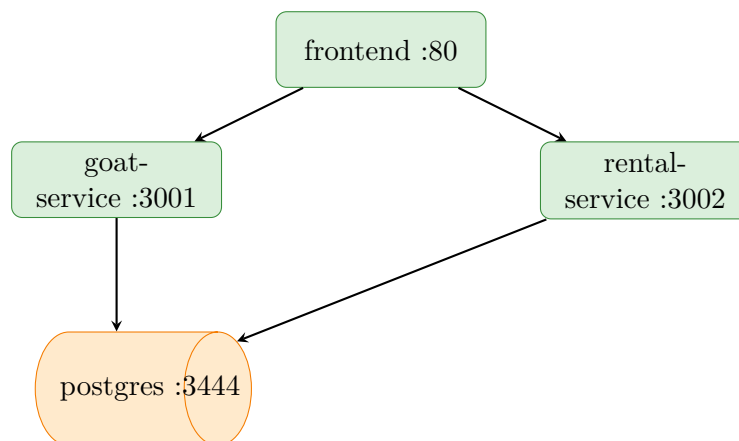
2.2.1 Goat Service

- **Controller** : `goat-controller.js` – Gestion des endpoints API pour les chèvres
- **Service** : `goat-service.js` – Logique métier (disponibilité, état des chèvres)
- **Data** : `goat-repository.js` – Accès PostgreSQL pour les données de chèvres

2.2.2 Rental Service

- **Controller** : `rental-controller.js` – Gestion des endpoints API pour les locations
- **Service** : `rental-service.js` – Logique métier (création, suivi, complétion des locations)
- **Data** : `rental-repository.js` – Accès PostgreSQL pour les données de locations

2.3 Architecture Docker



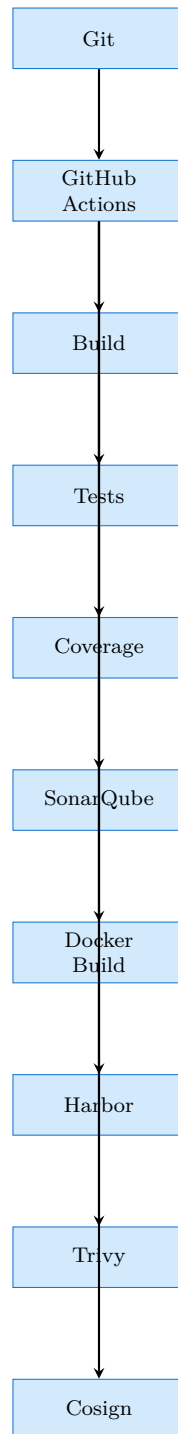
Chapitre 3

Pipeline CI (Continuous Integration)

3.1 Configuration GitHub Actions

Le pipeline CI est défini dans `.github/workflows/ci.yml` et s'exécute sur :

<https://github.com/GoatsSoftware/GoatUdder>



3.2 Étapes du pipeline

1. **Checkout** – Récupération du code source
2. **Setup Node.js** – Installation de Node.js 20
3. **Install dependencies** – `npm ci`
4. **Lint** – ESLint pour la qualité du code
5. **Tests** – Jest pour les tests unitaires
6. **Coverage** – Jest avec couverture ($\geq 80\%$ branches, fonctions, lines, statements)
7. **Build Docker** – Build et push vers Harbor
8. **SBOM** – Génération avec Syft
9. **Trivy** – Scan de vulnérabilités CRITICAL
10. **Cosign** – Signature des images Docker
11. **SonarQube** – Analyse de qualité du code

Chapitre 4

Rapport de Tests

4.1 Tests unitaires (couche Service)

4.1.1 Goat Service

```
goat-service/tests/goat-service.test.js – Tests unitaires du  
GoatService
```

4.1.2 Rental Service

```
rental-service/tests/rental-service.test.js – Tests unitaires du  
RentalService
```

4.2 Tests des Controllers (Mocks web avec Supertest)

4.2.1 Goat Controller

```
goat-service/tests/goat-controller.test.js – Tests avec supertest +  
jest.mock()
```

Endpoints testés :

- GET /goats – Retourne toutes les chèvres
- GET /goats/:id – Retourne une chèvre par ID
- GET /goats/available – Retourne les chèvres disponibles
- GET /goats/:id/available – Vérifie la disponibilité d'une chèvre

4.2.2 Rental Controller

<code>rental-service/tests/rental-controller.test.js</code>	–	Tests	avec
<code>supertest + jest.mock()</code>			

Endpoints testés :

- GET `/rentals` – Retourne toutes les locations
- GET `/rentals/:id` – Retourne une location par ID
- GET `/rentals/active` – Retourne les locations actives
- POST `/rentals` – Crée une location
- POST `/rentals/:id/complete` – Complète une location

4.3 Tests de la couche Data (Repository)

<code>goat-service/tests/goat-repository.test.js</code>	–	Tests	du
<code>GoatRepository</code>			

<code>rental-service/tests/rental-repository.test.js</code>	–	<i>[À ajouter : tests du RentalRepository]</i>	
---	---	--	--

Chapitre 5

Rapport de Couverture de Code

5.1 Seuils de couverture

Le pipeline CI impose les seuils minimums suivants :

Métrique	Seuil Minimum
Branches	$\geq 80\%$
Functions	$\geq 80\%$
Lines	$\geq 80\%$
Statements	$\geq 80\%$

5.2 Résultats de couverture

Les rapports de couverture sont générés par Jest (`lcov`) et uploadés comme artifacts GitHub Actions.

[Insérer screenshot des résultats de couverture GitHub Actions ici]

Chapitre 6

Rapport de Qualité

6.1 SonarQube

L'analyse de qualité est réalisée via SonarQube.

<https://sonarqube.techops.tf/dashboard?id=GoatUdder>

[Insérer screenshot du dashboard SonarQube ici]

Credentials :

- Username : [PLACEHOLDER_SONAR_USER]
- Password : [PLACEHOLDER_SONAR_PASSWORD]

6.2 ESLint

Le linting est exécuté à chaque build CI pour chaque service :

- goat-service/eslint.config.mjs
- rental-service/eslint.config.mjs

Chapitre 7

Captures d'écran des Google Labs

[Insérer screenshot des Google Labs ici]

Chapitre 8

Accès aux Ressources

8.1 Dépôt Git (GitHub)

Le repository est public et accessible à :

<https://github.com/GoatsSoftware/GoatUdder>

8.2 SonarQube

<https://sonarqube.techops.tf/dashboard?id=GoatUdder>

Credentials :

- Username : [PLACEHOLDER_SONAR_USER]
- Password : [PLACEHOLDER_SONAR_PASSWORD]

8.3 Harbor Registry (Images Docker)

<https://harbor.techops.tf/>

Credentials :

- Username : [PLACEHOLDER_HARBOR_USER]
- Password : [PLACEHOLDER_HARBOR_PASSWORD]

Chapitre 9

Conclusion

Le projet **GoatUdder** respecte l'ensemble des exigences DevOps :

- ✓ Dépôt Git structuré
- ✓ Pipeline CI avec GitHub Actions
- ✓ Architecture en couches (Data, Service, Controller)
- ✓ 2 services back conteneurisés (Goat Service, Rental Service)
- ✓ Tests de toutes les couches (unitaires, mocks web)
- ✓ Couverture de code $\geq 80\%$
- ✓ Qualité logicielle (SonarQube, ESLint)
- ✓ Frontend web
- ✓ Base de données PostgreSQL